

Q4 — Graph-Based Multi-Agent RAG

A bounded, inspectable workflow for local Malawi public-health retrieval • September 2026

1. Graph objective and topology

The graph turns the Q3 agent contracts into deterministic control flow. Nodes perform one responsibility and return structured state; conditional edges decide whether to answer, refine retrieval, filter unsafe evidence, or abstain. Local qwen3:8b can implement Verifier and Answerer, while Unicode normalization, layered injection detection, routing, retry limits, citation allow-lists, and audit events remain application controlled.

START	Evidence loop	Terminal path
User query	Retriever → Safety → Verifier	Answerer → END
User query	Retriever → Safety → Verifier → refined query → Retriever	Abstain → END after limit/failure
User query	Retriever → Safety removes injected chunks → Verifier	Answerer or bounded retry

Graph state

State field	Purpose	Written by
original_query, active_query, filters	Preserve user intent while allowing a focused retrieval rewrite.	START / Verifier route
retrieved_chunks, safe_chunks	Keep ranked candidates separate from policy-screened evidence.	Retriever / Safety
quarantined_ids	Audit which untrusted chunks were excluded.	Safety
verdict, approved_ids, missing_facts, refined_query	Make evidence decisions and next-query intent explicit.	Verifier
retry_count, max_retries	Guarantee termination; a retry is a return to retrieval.	Router / configuration
answer, cited_ids, abstain_reason, trace	Store final output and a reviewable execution record.	Answerer / Abstain / every node

Invariant. Only safe_chunks can reach verification, and only verifier-approved IDs can reach generation. The original query never changes. Each node appends a compact trace containing IDs and decisions rather than sensitive full passages.

2. Nodes and outputs

Node	Operation	Output / invariant
Retriever	Run hybrid BM25 + MiniLM retrieval for active_query and filters; retain ranks and component scores.	retrieved_chunks and query_used; no answer generation.
Safety	Normalize obfuscation; run word/compact patterns and optional semantic intent detection; quarantine by ID and emit safe audit metadata.	safe_chunks and quarantined_ids are disjoint; detector errors fail closed.
Verifier	Judge whether the safe set covers every requested fact. Return an approved allow-list and one focused refinement if evidence is incomplete.	verdict is sufficient, insufficient, or unsafe; approved_ids are a subset of safe IDs.
Answerer	Generate from approved chunks only, use a public-health-information tone, and cite exact chunk IDs.	A cited answer or deterministic abstention if citations are absent/invalid.
Abstain	End safely without calling generation.	Exact answer “cannot find in sources” plus internal reason.

Verifier contract

The Verifier receives the original question and safe evidence. It returns structured JSON: `verdict`, `approved_ids`, `missing_facts`, `refined_query`, and `confidence`. “Sufficient” means all requested factual parts are supported, not merely topically related. The refined query names the missing concept and cannot introduce a new user intent. Schema validation converts malformed output to “insufficient”.

Injection handling

Retrieved instructions are data, never commands. The Safety node applies security-only Unicode normalization, removes invisible formatting, maps a conservative set of homoglyph/leet variants, and checks both word-preserving and compact forms. A replaceable semantic detector handles intent outside fixed patterns and fails closed on detector errors. Flagged chunks are quarantined by ID before verification; raw malicious text is not written to audit logs.

Answer validation

The Answerer sees only approved chunks and the citation allow-list; the graph does not provide it with tool handles. A deterministic post-check requires a plain-text answer and at least one allow-listed `[chunk_id]` for factual output. The Q3 claim checker returns supported claims divided by total factual claims, so partial support receives a fractional score. A score below 0.90 routes to Abstain or one controlled revision, never an open-ended loop.

3. Conditional routing and termination

Condition after Verifier	Next edge	State update
sufficient AND <code>approved_ids</code> are a non-empty subset of safe IDs	Answerer	Pass only approved chunks.
insufficient AND <code>refined_query</code> exists AND <code>retry_count</code> < 2	Retriever	Increment <code>retry_count</code> ; set <code>active_query</code> to <code>refined_query</code> .
unsafe request	Abstain	Set reason; do not call Answerer.
No safe evidence and a retry remains	Retriever via Verifier	Use focused refinement; increment <code>retry_count</code> .
No refinement, malformed verdict, or retries exhausted	Abstain	Return exact fallback and END.
Answer citations missing or outside <code>approved_ids</code>	Abstain / END	Discard candidate; record validation failure.

Retry semantics

The initial retrieval is attempt 0. At most two returns from Verifier to Retriever are allowed, so the graph performs no more than three retrieval attempts. `retry_count` increments before each return edge. Answerer and Abstain are terminal. This bound prevents latency growth, cycling refinements, and repeated exposure to poisoned passages.

Failure handling and observability

Timeouts, classifier failures, invalid schemas, missing indexes, and empty results are represented as explicit state outcomes. Log node duration, attempt, retrieved/safe/approved IDs, quarantine detection layer, evidence-similarity score, route, verdict, citation result, fractional faithfulness, and terminal reason. Redact raw query or passage text when it may contain sensitive or malicious information.

Implementation mapping

The companion `snippets_q4.py` provides `GraphState`, node functions, conditional routing, and a dependency-free `run_graph` executor. Its injected retriever signature maps to the Q2 HybridRetriever; Verifier and Answerer can call local Ollama. The same node functions can later be registered in `LangGraph` without changing their state contracts or safety rules.